

**UNIVERSIDAD LAICA ELOY ALFARO DE MANABÍ****FACULTAD DE CIENCIAS DE LA VIDA Y TECNOLOGÍAS****TECNOLOGÍAS DE LA INFORMACIÓN****TEMA:****MODELOS DE CALIDAD DE SOFTWARE****INTEGRANTES:**

Héctor Matías Fernández Catagua

Winter Manuel Povea Bravo

ASIGNATURA:

Aplicaciones Web II

DOCENTE:

Ing. John Cevallos

JULIO DEL 2026

Héctor Fernández — Módulo Obligaciones y Multas**Qué aprendí**

Profundicé en la arquitectura en capas más allá de la teoría: entender por qué el service nunca debe depender de una implementación concreta de base de datos, sino de una interfaz, y cómo eso permite probar la lógica de negocio con fakes sin necesitar una base de datos real. También aprendí a modelar relaciones GORM (Belongs-To) entre Obligación/Multa y Usuario, a diseñar validaciones de negocio explícitas, y a escribir tests de integración que pasan por el middleware de autenticación y autorización con un JWT real, no simulado.

Qué haría distinto

Habría escrito los tests unitarios en paralelo con cada función del service, no al final, para no depender de la presión de los últimos días. También habría extraído la validación repetida entre crear y actualizar una obligación en un método privado compartido desde el inicio, en vez de duplicarla en dos lugares.

Próximos pasos

Automatizar el cálculo de mora según días de atraso desde la fecha de vencimiento, agregar un endpoint para marcar una obligación como pagada en una sola operación, y exponer un resumen de deuda total por residente.

Winter Povea — Módulo Parqueo (Vehículos, Visitas, Accesos)**Qué aprendí**

Entendí cómo diseñar un módulo con tres entidades relacionadas (Vehículos, Visitas, Accesos) manteniendo cada una con su propia responsabilidad dentro de la misma capa de storage. Trabajar con códigos QR para el control de entrada y salida de visitas me obligó a pensar en estados y transiciones (visita activa, visita con salida registrada), no solo en operaciones CRUD simples. También reforcé el manejo de roles: separar claramente qué puede hacer un admin frente a un residente en cada endpoint.

Qué haría distinto

Definiría el modelo de datos de Visitas y Accesos desde el inicio pensando en los estados posibles, en vez de ir agregando campos sobre la marcha a medida que surgían casos nuevos. Eso habría evitado un par de ajustes tardíos en el esquema.

Próximos pasos

Agregar notificaciones automáticas cuando una visita no registra salida dentro de un tiempo esperado, y un historial filtrable de accesos por vehículo o por residente.

Frank Montece — Módulo Área Social (Áreas comunes, Reservas, Notificaciones)**Qué aprendí:**

Aprendí a coordinar tres entidades donde una depende directamente de la disponibilidad de otra: una reserva solo tiene sentido si el área social y el horario están libres, lo que me llevó a pensar más en reglas de negocio que en simples inserciones a la base de datos. También con este módulo entendí mejor cómo las notificaciones pueden desacoplarse del flujo principal para no bloquear la respuesta al usuario.

Qué haría distinto

Habría definido con el grupo, desde el inicio del proyecto, un estándar único para el formato de fechas y horarios entre los tres módulos, ya que notamos inconsistencias menores al integrar Reservas con Obligaciones y Parqueo.

Próximos pasos

Agregar validación de solapamiento de horarios al reservar un área social, y una vista de disponibilidad semanal por área.

Reflexión grupal

Como equipo, el mayor aprendizaje fue la importancia de acordar convenciones compartidas (roles, formato de fechas, estructura de respuestas de error) antes de que cada integrante avanzara por separado en su módulo. La integración de los tres módulos bajo una misma autenticación JWT y un mismo esquema de roles (admin/residente) funcionó sin fricciones porque partimos de un middleware común desde el principio.

Como próximo paso conjunto, nos gustaría documentar un contrato de API más formal (OpenAPI/Swagger) para que la integración entre módulos, y con futuros consumidores del backend, dependa menos de la comunicación informal entre nosotros.